Spring AI Chat V2

A web-based AI chat application powered by Spring AI with dual provider support for Ollama and LM Studio, featuring real-time streaming responses, conversation persistence, multimodal image support, and a modern responsive UI.

Features

- **Dual AI Provider Support** -- Switch between Ollama and LM Studio backends from the UI
- **Multi-Model Support** -- Dynamically switch between any available model per conversation
- **Real-Time Streaming** -- Responses stream token-by-token via Server-Sent Events (SSE)
- **Conversation Management** -- Create, browse, and delete conversations with full message history
- **Image Support** -- Send images (PNG, JPG, GIF, WebP) to multimodal models like Gemma 4 for visual analysis
- **File Attachments** -- Attach code and text files (20+ formats) for AI analysis
- **Markdown Rendering** -- Full Markdown support with syntax-highlighted code blocks and copy buttons
- **Dark/Light Theme** -- Toggle between themes with persistent preference
- **Workspace File Tools** -- Models can create project files and folders in a sandboxed workspace, with a built-in file browser panel
- **Dual Tool Strategy** -- Native `@Tool` calling for capable models (e.g., Gemma 4), automatic prompt-parsing fallback for all others
- **Response Archive** -- Every model response is automatically saved as a `.md` file in `workspace/.archive/`, guaranteeing output is never lost regardless of tool or parsing success
- **Manual File Extraction** -- An "Extract Files" button in the workspace panel re-scans all assistant messages and extracts code blocks into workspace files on demand
- **Automatic Memory Management** -- Switching providers and stopping the app both unload models from RAM/VRAM automatically (via `@PreDestroy` on shutdown)
- **Smart Acknowledgment Detection** -- Recognizes short responses like "thanks" or "ok" and avoids repeating previous content
- **Auto-Titling** -- Conversations are automatically titled from the first message
- **Model Management** -- Browse available models from both providers, check connection status, and pull new Ollama models from the config page
- **Responsive Design** -- Works on desktop and mobile with collapsible sidebar

Tech Stack

Layer	Technology
Backend	Spring Boot 3.4.2, Java 17
AI Integration	Spring AI 1.1.4 (Ollama + OpenAI-compatible/LM Studio)
Frontend	Thymeleaf, HTMX 2.0.4, vanilla JavaScript
Database	H2 (file-based, embedded)
Markdown	marked.js + DOMPurify
Build	Maven

Prerequisites

- **Java 17** or higher
- **Maven 3.8+**
- **Ollama** installed and running on `http://localhost:11434` (for Ollama provider), or a RunPod GPU pod with Ollama (see RunPod Cloud GPU)
- **LM Studio** with local server enabled on `http://localhost:1234` (for LM Studio provider)
- At least one model available in either provider (e.g., `ollama pull mistral`)

Getting Started

1. Install Ollama

Follow the instructions at ollama.com to install Ollama for your platform.

2. Pull a Model

```
ollama pull mistral
```

For image/multimodal support:

```
ollama pull gemma4:e2b
```

3. Run the Application

```
mvn spring-boot:run
```

The application starts at **http://localhost:8080**.

4. Start Chatting

1. Select a provider (**Ollama** or **LM Studio**) from the sidebar dropdown
2. Select a model from the model dropdown (updates dynamically based on provider)

3. Click **+ New Chat**
4. Type a message and press Enter

RunPod Cloud GPU

The app supports running Ollama on a RunPod cloud GPU, letting you use large models without local GPU hardware. Your machine only runs the lightweight web server -- all AI inference happens on the remote GPU.

Setup

1. Create a RunPod account and deploy a GPU Pod (L40S 48GB, A100 80GB, etc.)
2. SSH into the pod or use the Web Terminal and start Ollama:

```
OLLAMA_HOST=0.0.0.0 ollama serve &
ollama pull mistral:latest
```

3. Note your pod's proxy URL from the RunPod dashboard (under **Connect**):

```
https://<your-pod-id>-11434.proxy.runpod.net
```

4. Run the app with the `runpod` profile:

```
mvn spring-boot:run -Dspring-boot.run.profiles=runpod
```

Configuration

Update `src/main/resources/application-runpod.properties` with your pod URL:

```
spring.ai.ollama.base-url=https://<your-pod-id>-11434.proxy.runpod.net
```

Running Modes

Mode	Command	AI runs on
Local	`mvn spring-boot:run`	Your machine (localhost:11434)
RunPod	`mvn spring-boot:run -Dspring-boot.run.profiles=runpod`	RunPod cloud GPU

Important Notes

- Ollama must be started with `OLLAMA_HOST=0.0.0.0` on the pod so the RunPod proxy can reach it
- Pull models on the pod, not locally -- they need to exist on the remote Ollama instance
- Pods with ephemeral storage lose models on restart; use persistent volumes if needed

- Stop your pod when not in use to avoid charges

LM Studio Integration

The app supports LM Studio as an alternative AI provider via its OpenAI-compatible API.

Setup

1. Download and open LM Studio
2. Download a model from the LM Studio model browser
3. Go to the **Developer** tab and click **Start Server** (default port: 1234)
4. Make sure a model is **loaded** (active) in the server

Usage

1. Select **LM Studio** from the provider dropdown in the sidebar
2. The model list updates to show models available in LM Studio
3. Select a model and start a new chat

Features with LM Studio

- **Streaming responses** -- Same real-time token-by-token streaming as Ollama
- **Native tool calling** -- Models with tool/function calling support (shown in LM Studio's model skills) can use the workspace file tools natively
- **Provider per conversation** -- Each conversation remembers its provider, so you can have Ollama and LM Studio chats side by side

Notes

- LM Studio models are managed through the LM Studio GUI (no pull functionality from the app)
- The config page shows connection status and available models for both providers
- LM Studio must have its local server running **and** a model loaded for the app to connect. Loading a model in LM Studio's chat is **not** enough -- you must also go to the **Developer** tab and click **Start Server** to expose the API on port 1234

Image Support

To send an image to a multimodal model (e.g., Gemma 4):

1. Select a vision-capable model from the dropdown
2. Click the attachment button (paperclip icon)
3. Select an image file (PNG, JPG, GIF, or WebP, up to 5MB)
4. Type your prompt (e.g., "Describe this image") and send

The image is displayed inline in the chat and sent to the model via Spring AI's multimodal Media API.

GPU Acceleration

For faster image inference, you can create a GPU-accelerated model variant using an Ollama Modelfile. This offloads a portion of the model layers to your GPU:

```
# Create a Modelfile
echo 'FROM gemma4:e2b
PARAMETER num_gpu 20' > Modelfile.gemma4-gpu

# Build the model variant
ollama create gemma4-gpu -f Modelfile.gemma4-gpu
```

Adjust `num_gpu` based on your available VRAM. The original model remains untouched -- you can select either variant from the app's model dropdown.

For best results, enable flash attention in the Ollama service configuration:

```
sudo systemctl edit ollama
```

Add:

```
[Service]
Environment="OLLAMA_FLASH_ATTENTION=1"
```

Then restart:

```
sudo systemctl daemon-reload
sudo systemctl restart ollama
```

Memory Management & Dual Provider Usage

Both Ollama and LM Studio load models into system RAM (and VRAM if using GPU). Running both providers simultaneously with models loaded requires sufficient memory.

Hardware Considerations

Setup	Minimum RAM	Notes
Single provider (Ollama **or** LM Studio)	8-16 GB	Depends on model size (7B ~ 4-8 GB, 13B ~ 8-14 GB)
Both providers simultaneously	24-32+ GB	Two models loaded at once plus OS and application overhead

On systems with limited memory (e.g., 16 GB), running both providers with models loaded at the same time can cause swapping or out-of-memory issues.

Automatic Model Unloading

The app automatically manages memory in three scenarios:

- **Switching provider to LM Studio** -- All Ollama models are unloaded via `keep_alive:`

`0`

- **Switching provider to Ollama** -- All LM Studio models are unloaded via the developer API
- **App shutdown (Ctrl+C / SIGTERM / container stop)** -- A Spring `@PreDestroy` hook unloads loaded models from both providers so RAM is freed immediately, instead of waiting for Ollama's `keep_alive` TTL to expire

This ensures only **one provider's models** occupy memory at a time during use, and that models don't linger after the app is stopped. Note: `kill -9` or a JVM crash will bypass the shutdown hook -- in that case Ollama's `keep_alive` TTL still applies.

You can also manually unload models via the API:

```
# Unload all Ollama models
curl -X POST http://localhost:8080/config/ollama/unload

# Unload all LM Studio models
curl -X POST http://localhost:8080/config/lmstudio/unload
```

> **Note:** If you have enough RAM/VRAM to run both providers simultaneously, the auto-unload is harmless -- the model simply reloads on the next request. Each conversation remembers its provider, so you can have Ollama and LM Studio chats side by side if your hardware supports it.

Response Archive

Every model response is automatically saved as a Markdown file in `workspace/.archive/`, organized by conversation. This provides a guaranteed record of all model output, independent of whether native tool calling or prompt parsing succeeded.

Archive Structure

```
workspace/.archive/
└── conversation-1/
    ├── 20260410_213045_mistral_latest.md
    ├── 20260410_213112_mistral_latest.md
    └── ...
```

Each file includes metadata headers (model name, timestamp, conversation ID) followed by the full response text.

Manual File Extraction

If a model generated code blocks in a conversation where Tools were not enabled, you can retroactively extract them:

1. Enable the **Tools** checkbox in the chat header to open the workspace panel
2. Click the green **Extract Files** button in the workspace panel header
3. The app scans all assistant messages in the current conversation and extracts any code blocks with file paths into the workspace

This works with 4 different code block formats that models commonly use, including labeled markdown blocks, language:path fences, and first-line comment paths.

Workspace File Tools

The app can instruct models to create real project files in a sandboxed `./workspace/` directory. This lets you ask a model to scaffold an entire project and then browse, preview, and download the generated files.

How to Use

1. Open a chat and check the **Tools** checkbox in the chat header
2. A workspace file browser panel opens on the right
3. Ask the model to create a project, e.g.:
    - *"Create a Spring Boot hello world project with all necessary files"*
    - *"Build a Python Flask REST API with a Dockerfile"*
4. Files appear in the workspace panel after the response completes
5. Click any file to preview, download, or delete it

How It Works (Dual Strategy)

The app picks a strategy per-request based on the model's advertised capabilities:

1. Native Tool Calling -- For Ollama models that advertise the `tools` capability (e.g., Gemma 4). The server uses a tool-oriented system prompt and streams with `@Tool` methods bound (`createFolder`, `writeFile`, `readFile`, `listFiles`). Spring AI handles the tool-call round-trip inside the reactive stream, so long model-load times never trip a non-streaming read timeout.

2. Prompt-Parsing Fallback -- For non-tool Ollama models and all LM Studio models. The system prompt asks the model to label code blocks with file paths:

````
**`src/main/java/App.java`**
```java
// code here
```
````

After streaming completes, the server parses these labeled code blocks (4 pattern strategies) and writes the files. This works with **any model** since it only requires standard markdown output.

The tool-oriented prompt explicitly allows general conversation, image analysis, and code explanation -- tools are only invoked when the user asks for file creation. Vision still works with Tools mode enabled.

Console logs show which strategy fired:

- `[Native Tool] Written: <path>` + `Native tool calling succeeded: N tool calls` -- `@Tool` methods were invoked
- `[Prompt-Parse] Created N files` -- extracted from markdown after streaming

Workspace Configuration

Property	Default	Description
`workspace.root`	`./workspace`	Directory where models create files (sandboxed, path traversal prevented)

All file operations are restricted to the workspace directory. The workspace is created automatically on startup.

File Attachments

Supported text/code file formats (up to 512KB):

`.txt` `.md` `.java` `.py` `.js` `.ts` `.html` `.css` `.xml` `.json` `.yaml` `.yml` `.properties` `.sql` `.sh` `.csv` `.log` `.c` `.cpp` `.h` `.go` `.rs` `.rb` `.php` `.kt` `.swift` `.scala` `.gradle` `.toml`

File contents are embedded directly into the message text for the model to analyze.

Configuration

Application Properties

Property	Default	Description
`spring.ai.ollama.base-url`	`http://localhost:11434`	Ollama server URL (overridden by `runpod` profile)
`spring.ai.ollama.chat.options.model`	`mistral:latest`	Default Ollama model
`spring.ai.openai.base-url`	`http://127.0.0.1:1234`	LM Studio server URL (OpenAI-compatible)
`spring.ai.openai.api-key`	`lm-studio`	API key placeholder (LM Studio ignores this)
`spring.datasource.url`	`jdbc:h2:file:./data/chatdb`	Database file location
`server.tomcat.max-http-form-post-size`	`10MB`	Max POST body size (for image uploads)
`workspace.root`	`./workspace`	Workspace directory for file tools

Config Page

Visit **http://localhost:8080/config** to:

- Check Ollama and LM Studio connection status
- View available models from both providers

- Pull new models from the Ollama registry

Project Structure

```
src/main/
├── java/com/example/chat/
│   ├── ChatApplication.java            # Entry point
│   ├── controller/
│   │   ├── ChatController.java         # Chat & streaming endpoints
│   │   ├── ConfigController.java       # Ollama config & model management
│   │   └── WorkspaceController.java    # Workspace file browsing & download
│   ├── service/
│   │   ├── ChatService.java            # Chat logic, prompt building, dual-
provider streaming
│   │   ├── OllamaService.java          # Ollama API client (model listing,
pull, unload)
│   │   ├── LmStudioService.java        # LM Studio API client (OpenAI-
compatible, unload)
│   │   └── ResponseArchiveService.java # Guaranteed response archiving to
workspace
│   ├── tools/
│   │   └── FileTools.java              # @Tool methods + prompt-parsing
fallback
│   ├── model/
│   │   ├── Conversation.java           # Conversation entity
│   │   └── ChatMessage.java            # Message entity (text + image data)
│   └── repository/
│       ├── ConversationRepository.java
│       └── ChatMessageRepository.java
└── resources/
    ├── application.properties
    ├── templates/
    │   ├── index.html                  # Main chat UI
    │   ├── config.html                 # Settings page
    │   └── fragments/
    │       ├── chat.html
    │       └── sidebar.html
    └── static/
        ├── js/app.js                   # Frontend logic
        └── css/style.css               # Theming & layout
```

How It Works

1. **User sends a message** -- saved to H2 database via POST, then SSE streaming begins
2. **ChatService builds the prompt** -- loads full conversation history, applies system prompt (tool-aware or standard), attaches image Media objects if present
3. **Spring AI calls the selected provider** (Ollama or LM Studio via OpenAI-compatible API) -- everything flows through a reactive `Flux<String>` stream, with `@Tool` methods bound when Tools are enabled so capable models can invoke them mid-stream
4. **Frontend renders in real-time** -- SSE events update the DOM, Markdown is parsed and sanitized on each chunk. Client disconnects are detected and the upstream Flux is cancelled so tokens aren't generated for nobody
5. **File extraction** -- when Tools are enabled, `@Tool` calls fire during streaming for capable models; for everything else the server parses labeled code blocks from the final response (4

pattern strategies) and writes files to the workspace

6. **Response is archived** -- every response is saved as a `.md` file in `workspace/.archive/` for guaranteed persistence
7. **Response is persisted** -- the full assistant response is saved to the H2 database and the workspace panel refreshes
8. **Memory management** -- switching providers unloads the previous provider's models, and a `@PreDestroy` hook unloads all loaded models on app shutdown

License